

Test-Time Adaptation: adapting to the unknown on the fly

Gustavo A. Vargas Hakim

Collaborators



Christian Desrosiers

Professor



Ismail Ben Ayed

Professor



David Osowiechi

PhD Student



Gustavo Vargas

PhD Student

ILLS



ILLS



CentraleSupélec



ILLS
International Laboratory
on Learning Systems



Le génie pour l'industrie



What us humans can do, and machines can't

As simple as it can be, humans excel at detecting things:



A **photo** of
a horse



An **artwork** of a
horse



A **cartoon** of a
horse



A **sketch** of a
horse

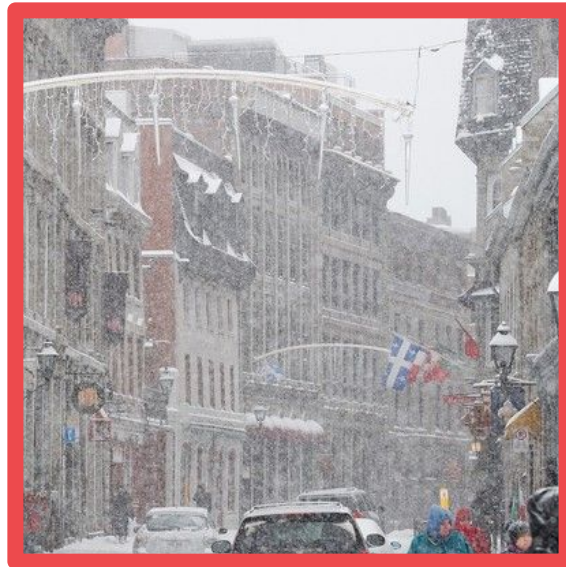
*Images from PACS dataset.

What us humans can do, and machines can't

Even weather conditions can make it challenging:



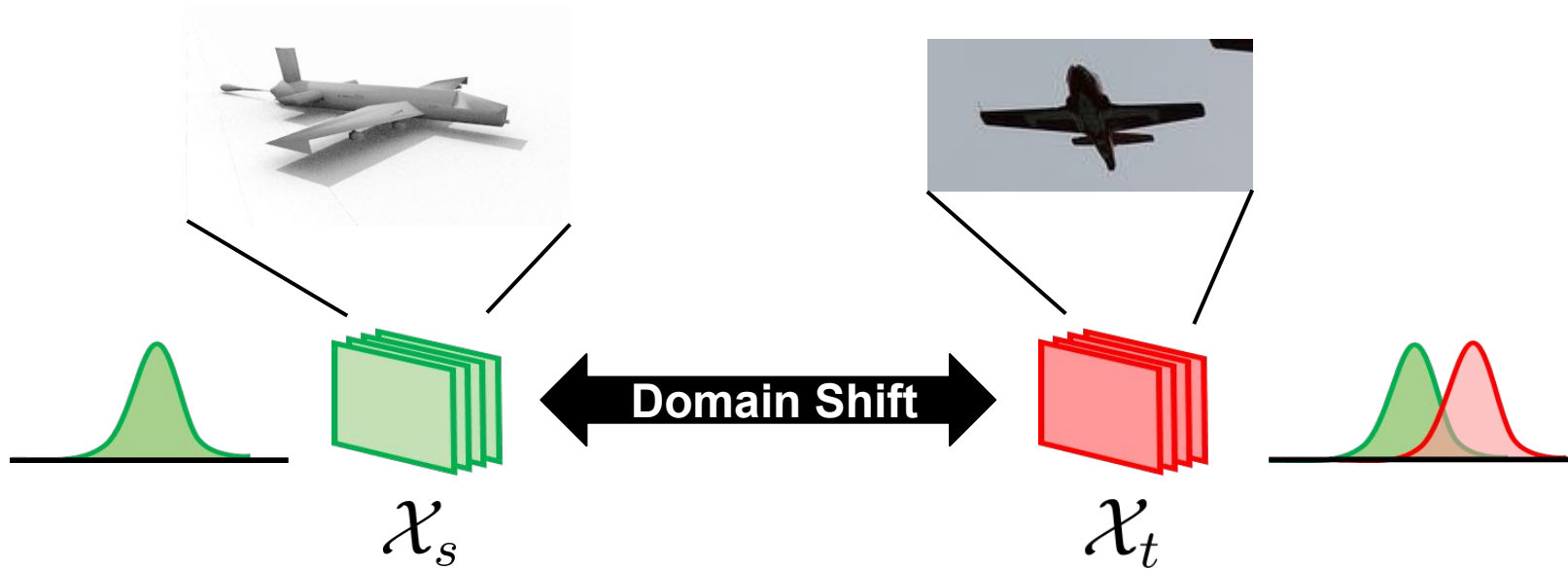
**Summer
for training**



**Winter when
testing**

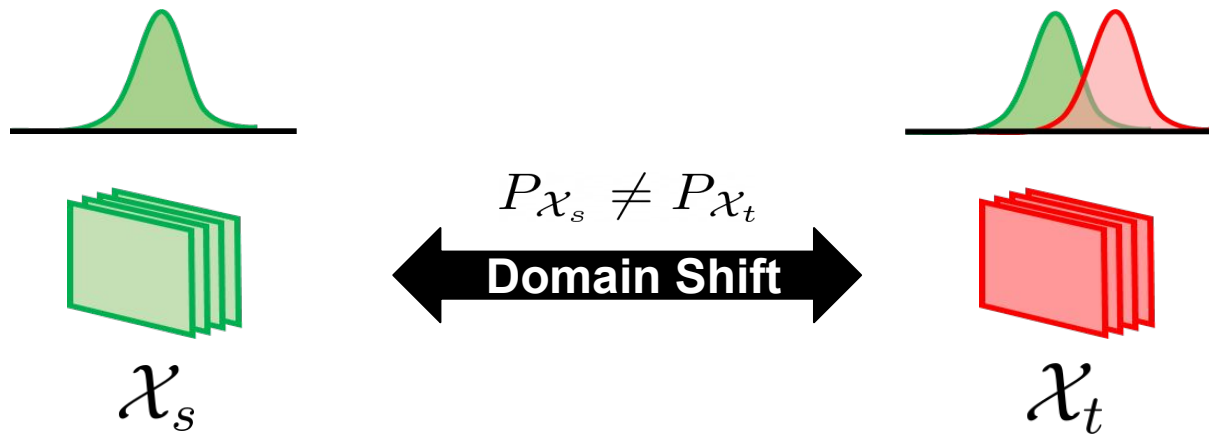
The problem of domain shift

Source data and target data are usually not in the same domain:



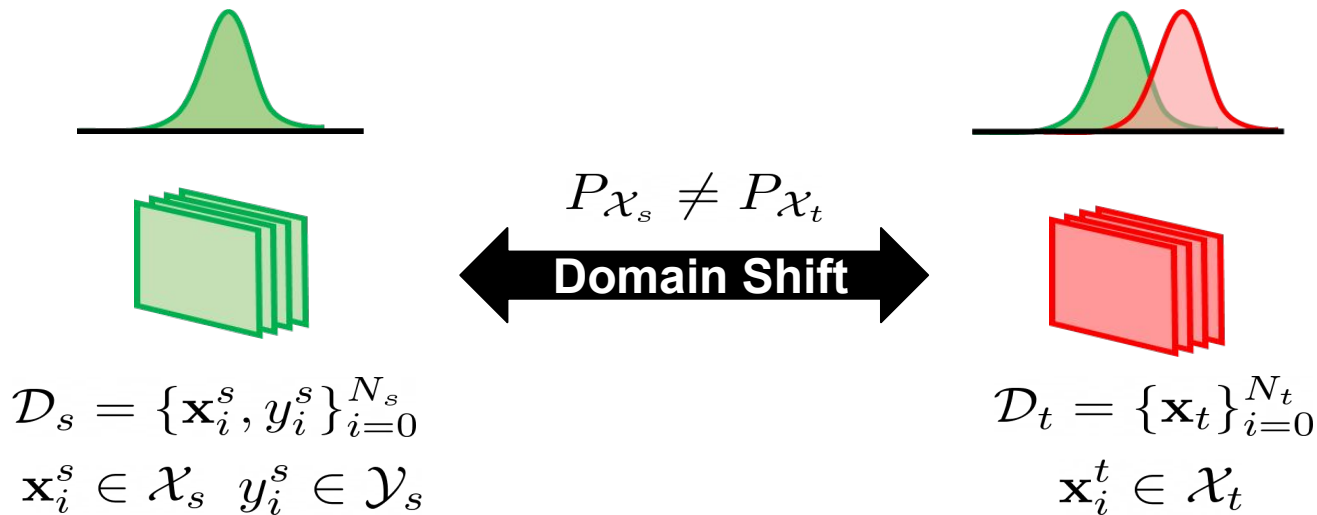
The problem of domain gap

Source data and target data are usually not in the same domain:



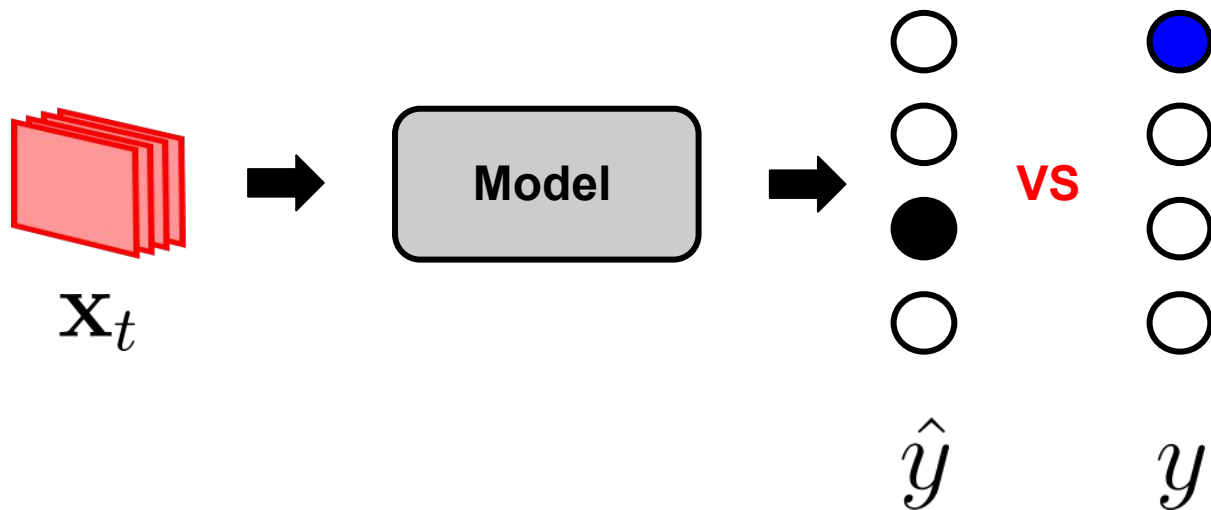
The problem of no supervision

The target data is also not supervised:



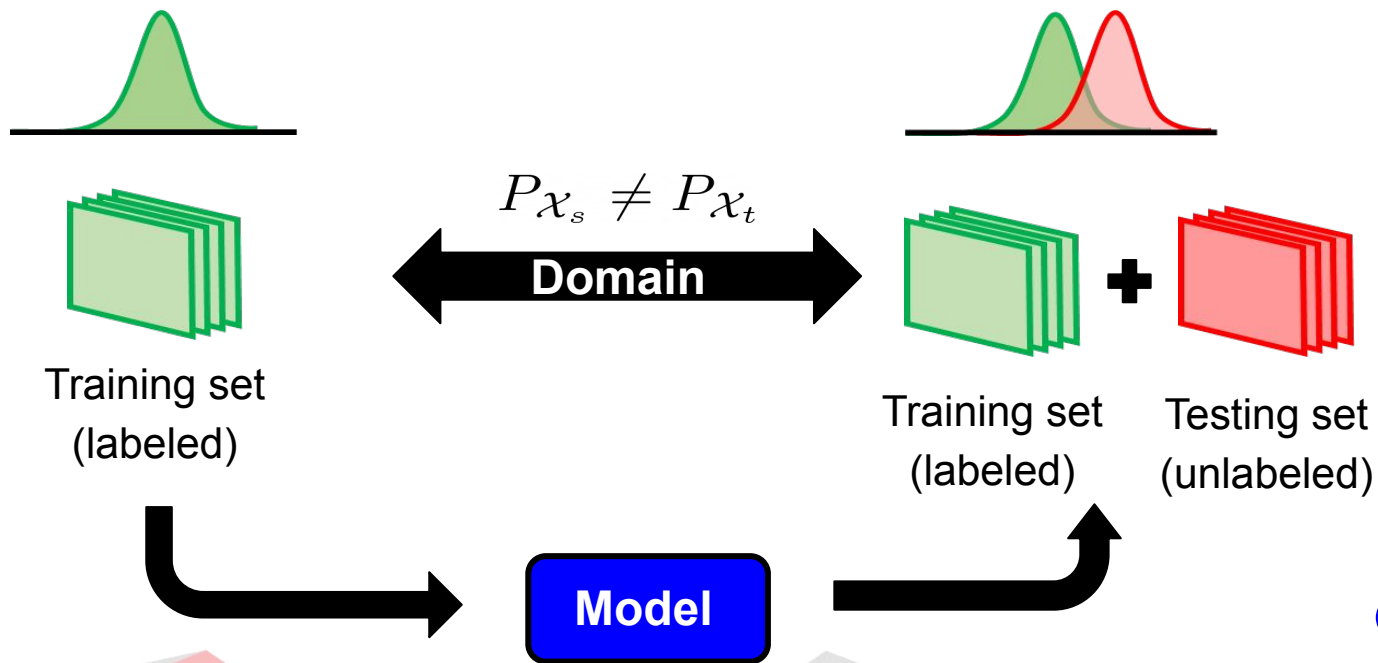
Why is it important?

Most models will fail at test-time:



Domain Adaptation

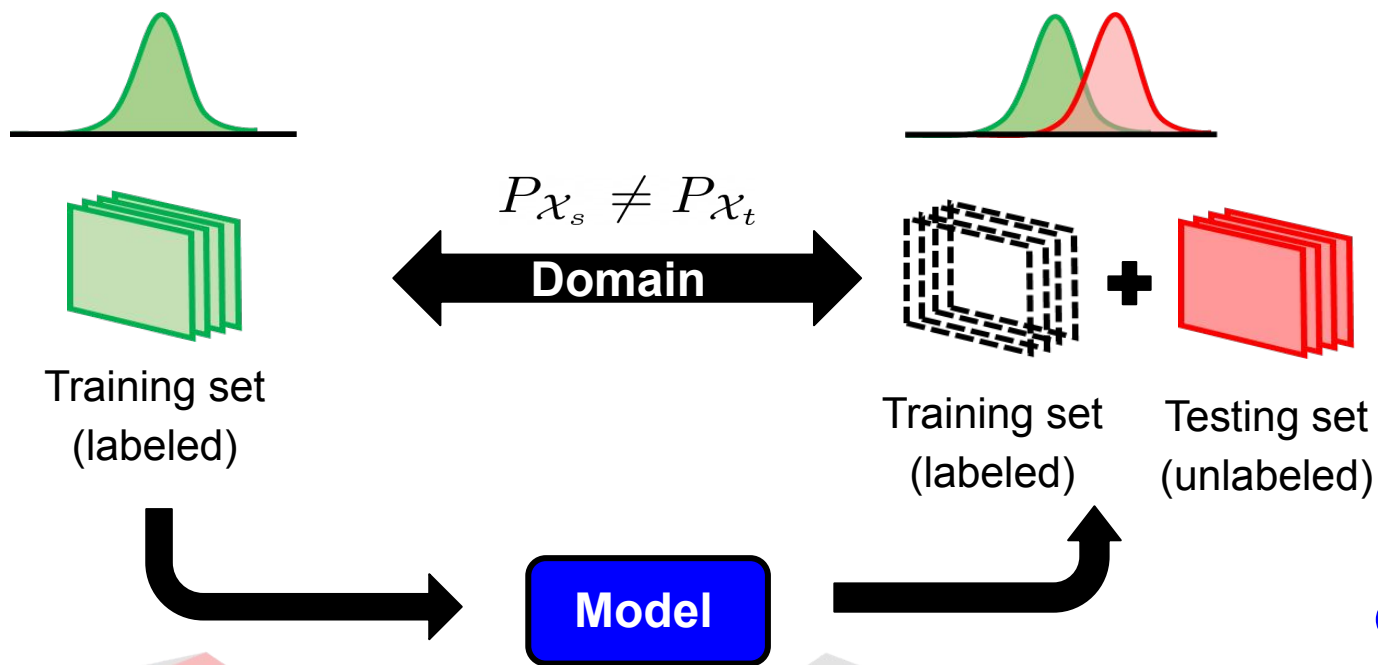
Most models will fail at test-time:



● Active

Test-Time Adaptation

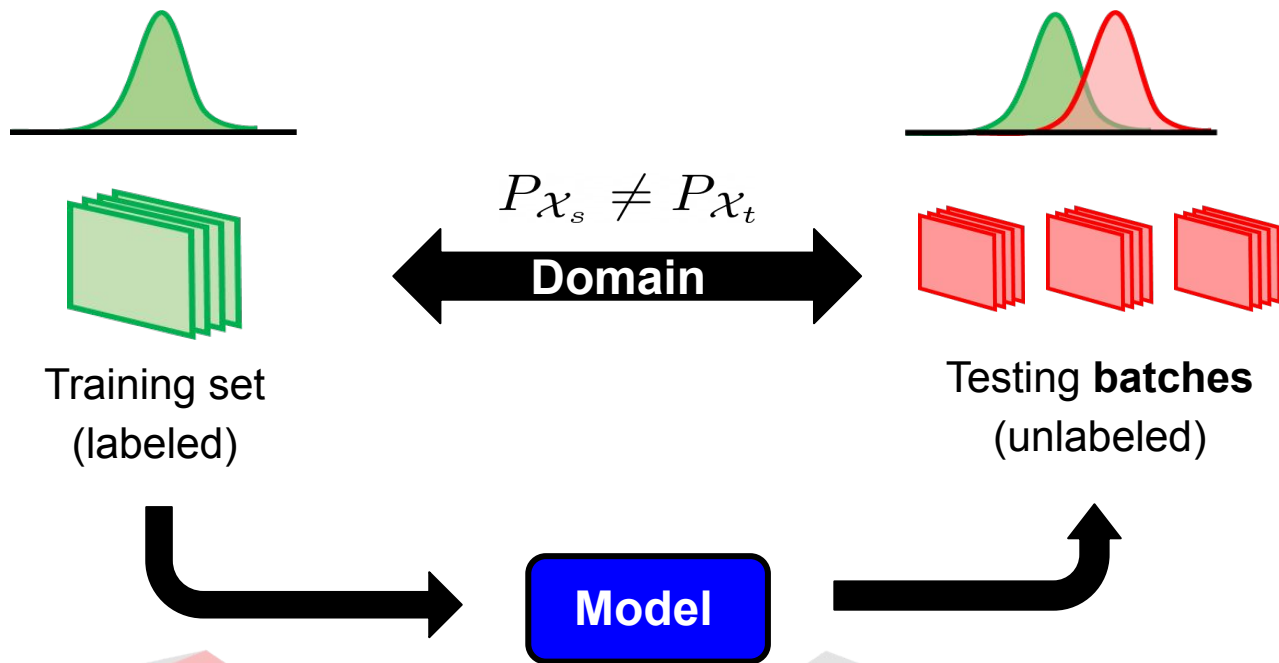
Most models will fail at test-time:



Active

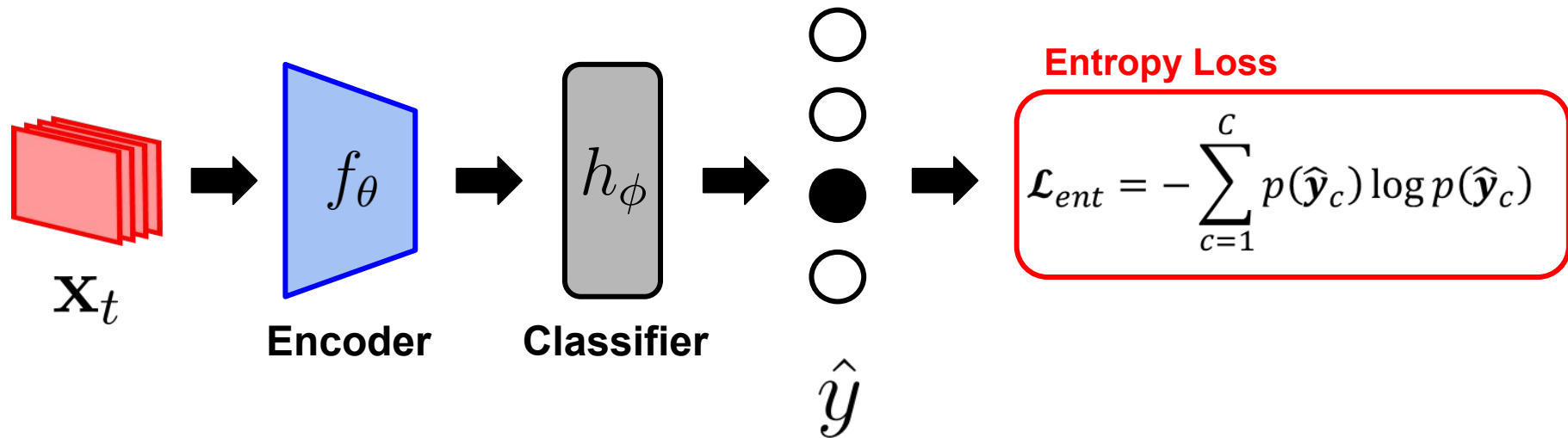
Test-Time Adaptation

Most models will fail at test-time:



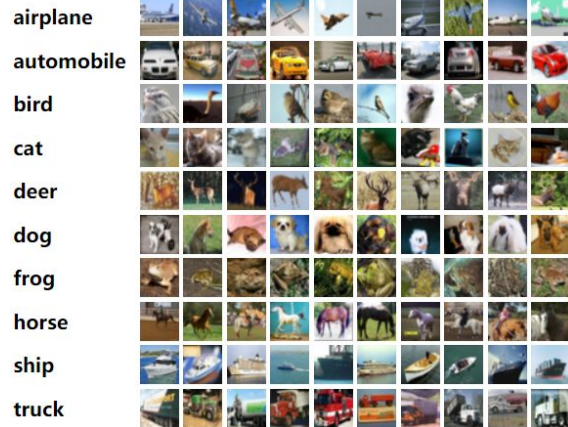
TENT

Entropy minimization to update batch norm affine parameters:



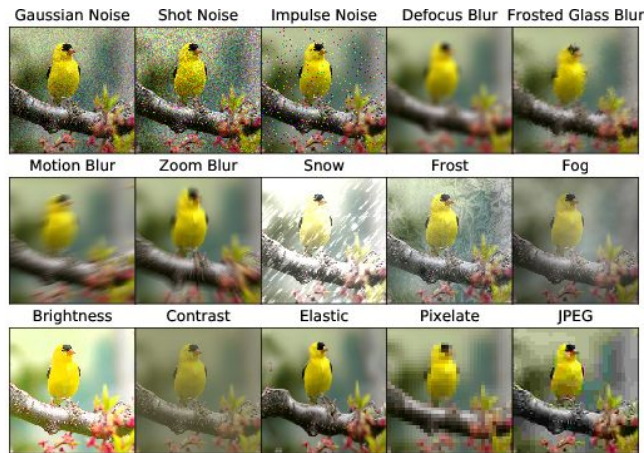
Experimental Setup

Common corruptions:



CIFAR-10/100

Source data



CIFAR-10/100-C

Target data

10k images,
15 corruptions,
5 severity levels

Experimental Setup

Sim-to-real domain shift:



VisDA-C (3D)

Source data

152,397 images
of 12 classes



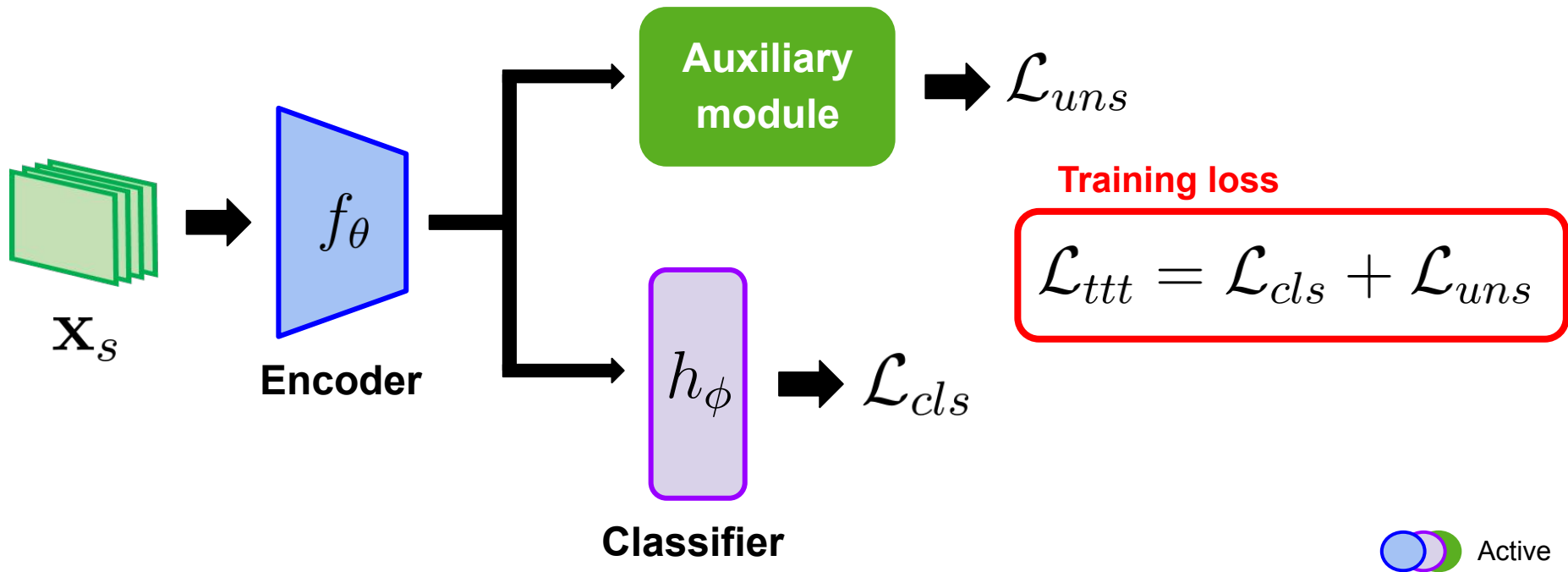
VisDA-C (real)

Target data

72,327 video
frames of the
same 12
classes

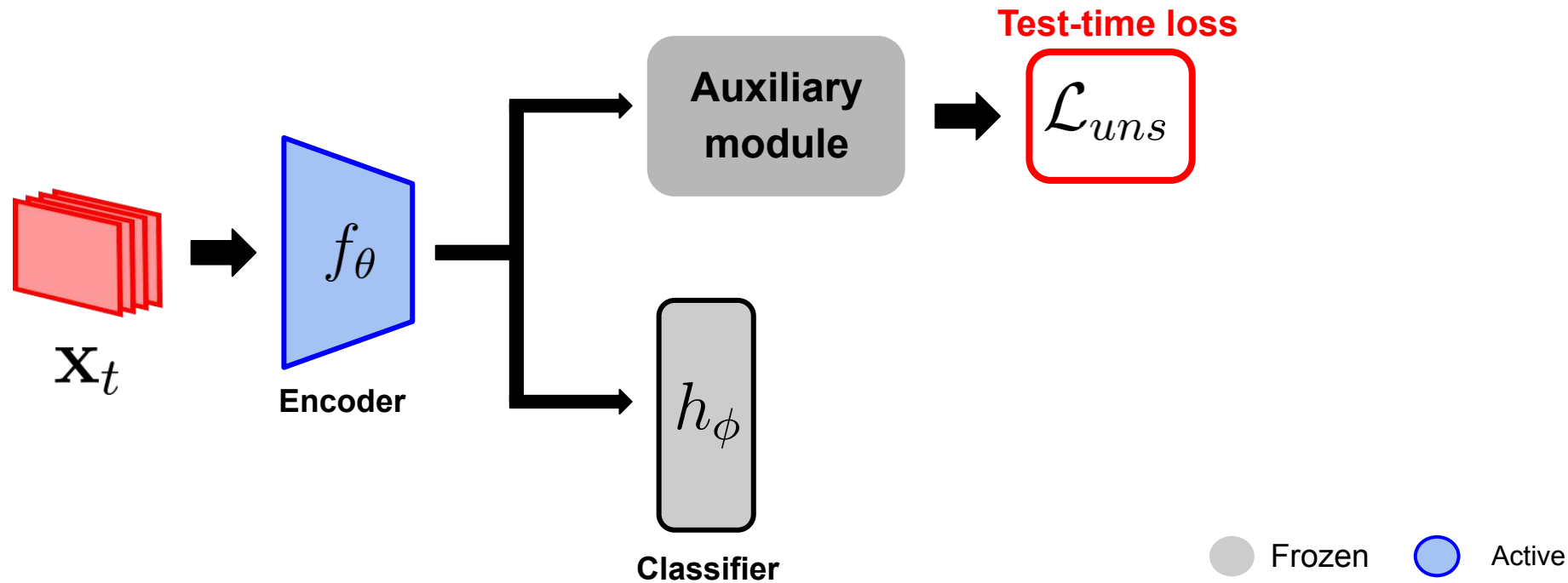
Test-Time Training

A special solution to TTA. We require a special **training phase**:



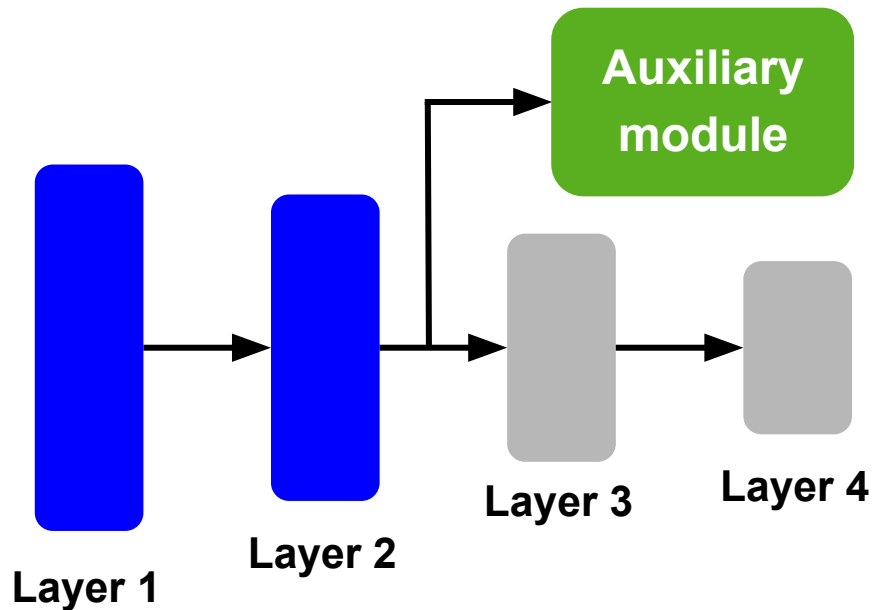
Test-Time Training

During the **testing phase**, we concentrate only on the unsupervised task:



Interesting remark on TTT

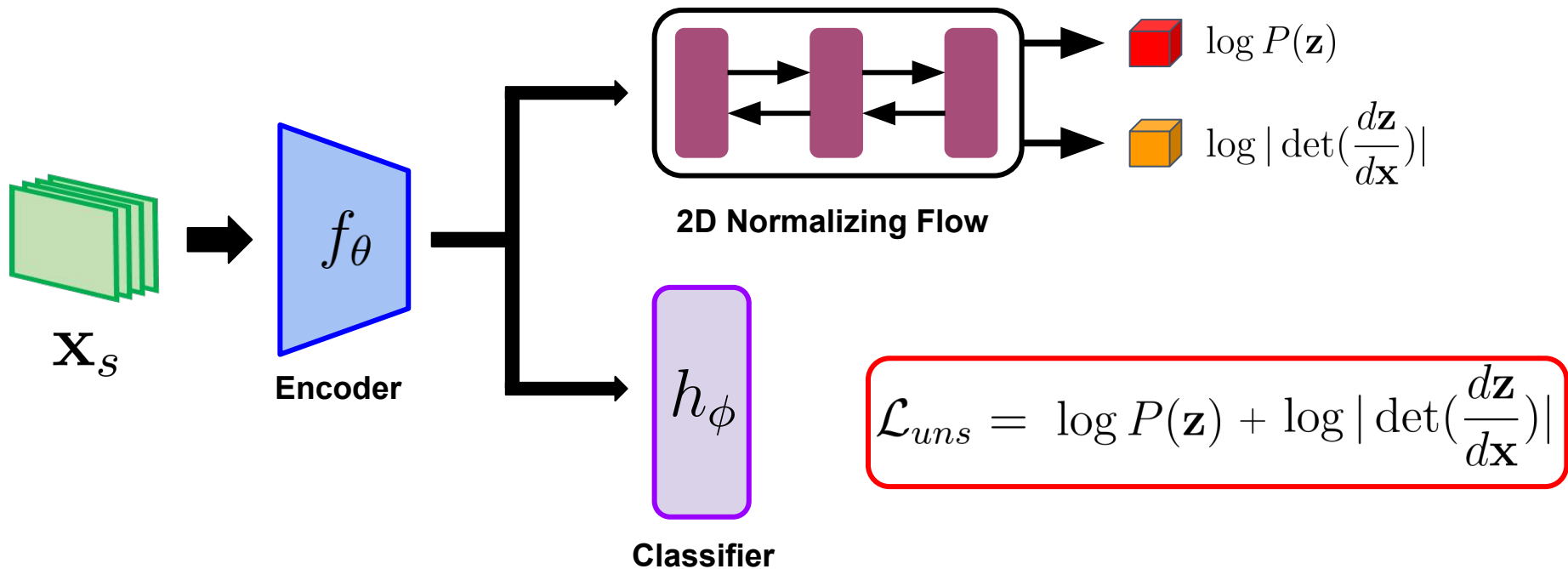
Usually, we adapt only the first layers of the encoder:



TTTFlow: Unsupervised Test-Time Training with Normalizing Flow. WACV 2023

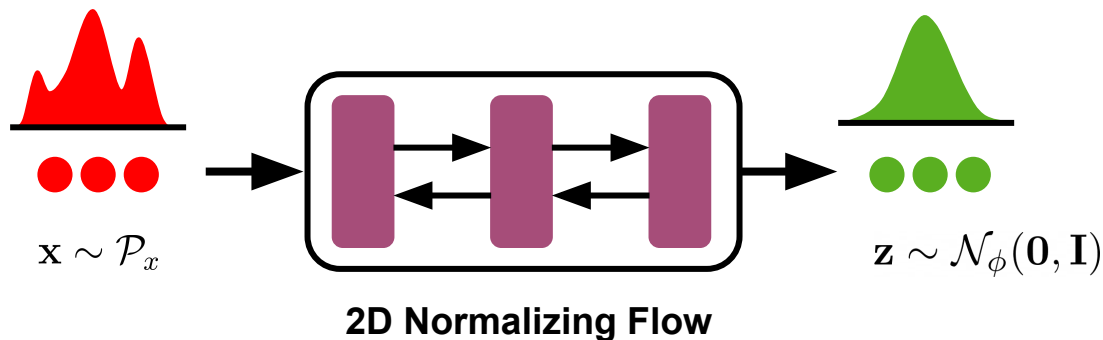
TTTFlow

A Normalizing Flow learns the distribution of source features. **Training:**



TTTFlow

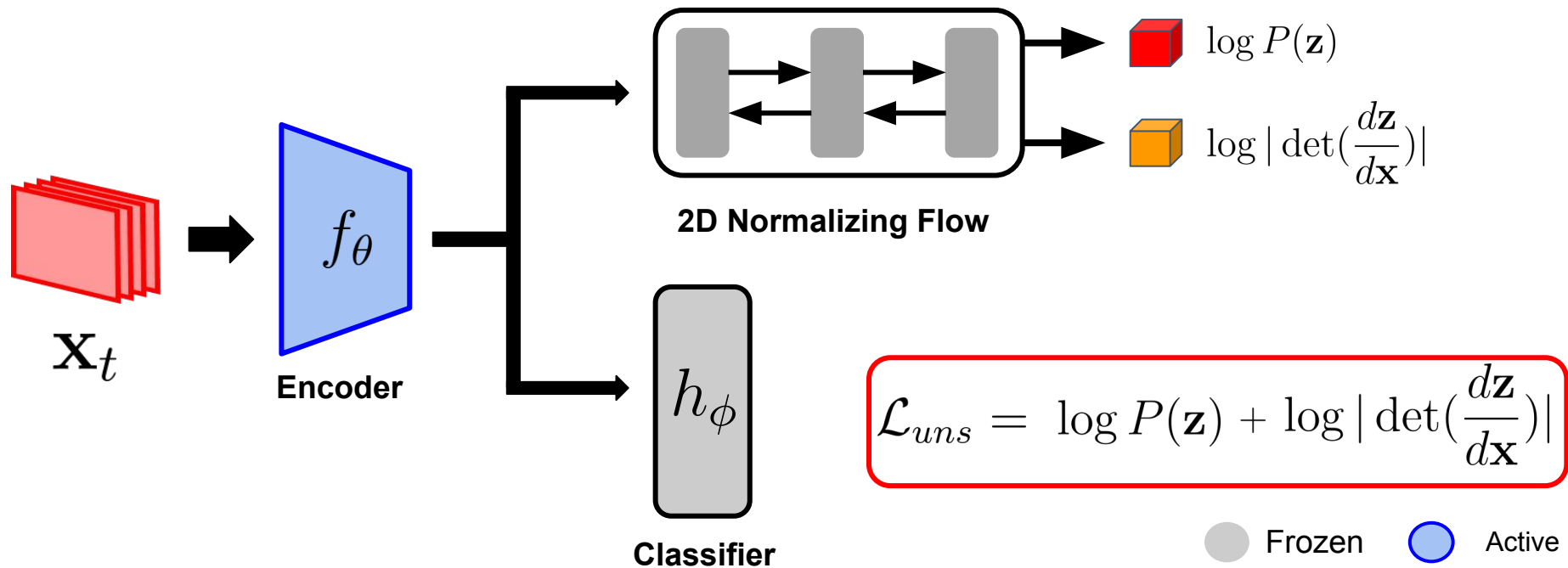
A Normalizing flow is a bijective and differentiable function that transforms any (unknown) distribution into a Gaussian distribution:



We can generate new samples from it, or in this case, measure the exact likelihood of any sample.

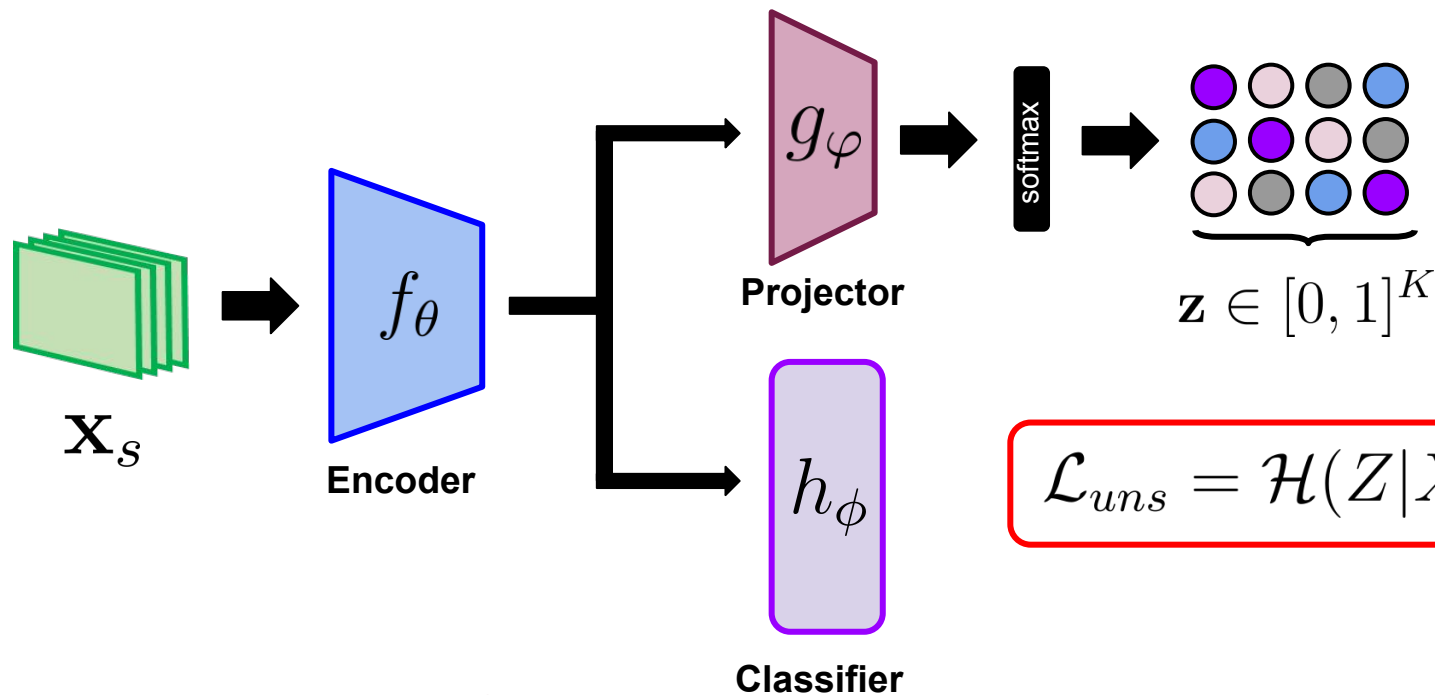
TTTFlow

A Normalizing Flows learns feature source distribution. **Testing:**



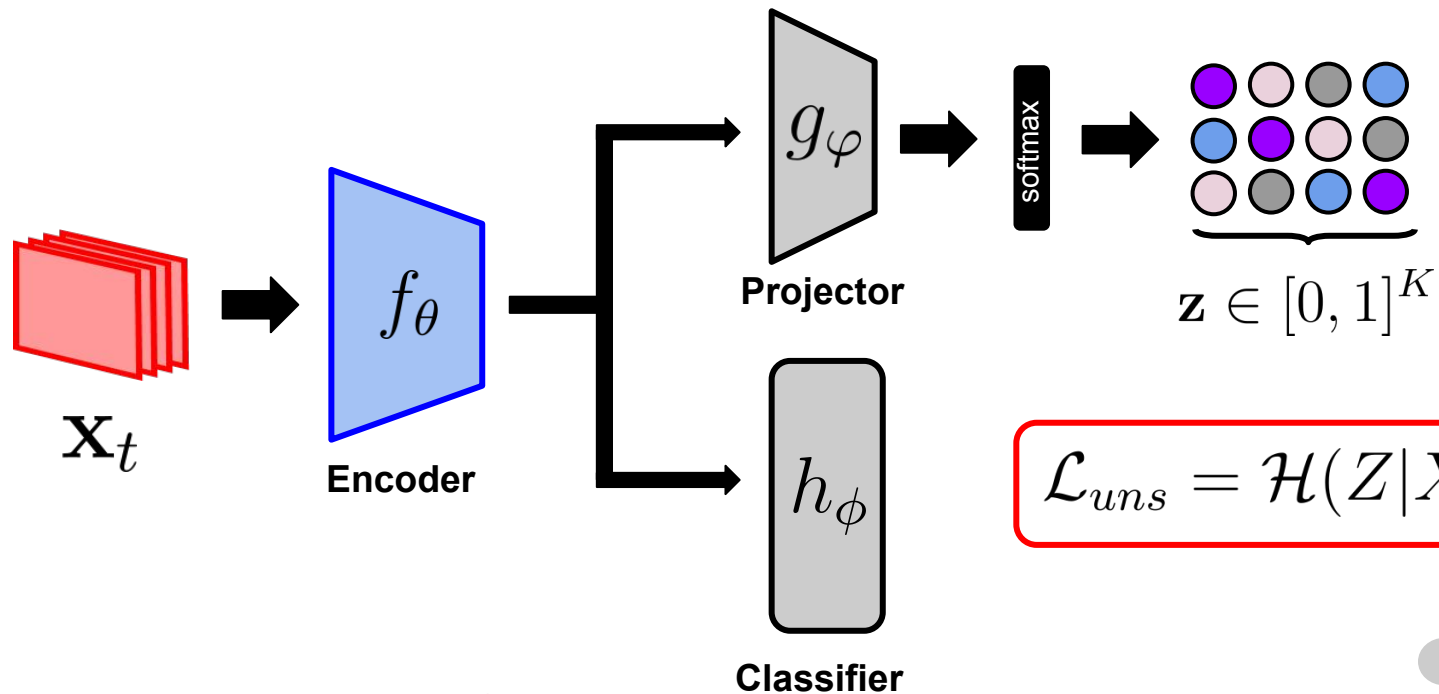
ClusT3: Information Invariant Test Time Training. ICCV 2023

Source distribution is learned through a discrete clustering task. **Training:**



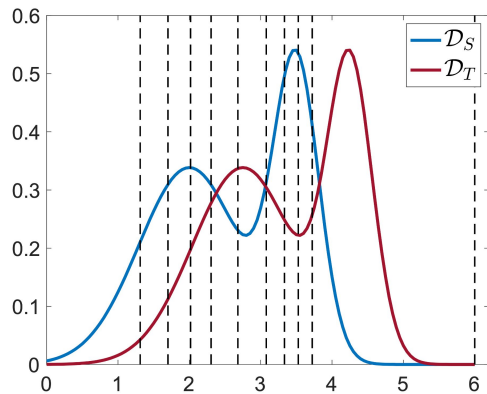
$$\mathcal{L}_{uns} = \mathcal{H}(Z|X) - \mathcal{H}(Z)$$

Source distribution is learned through a clustering task. **Testing:**



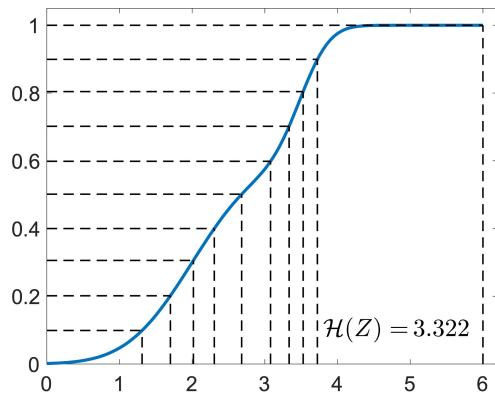
ClusT3: toy visualization with 10 classes

We enforce an even partition in the target domain's CDF:



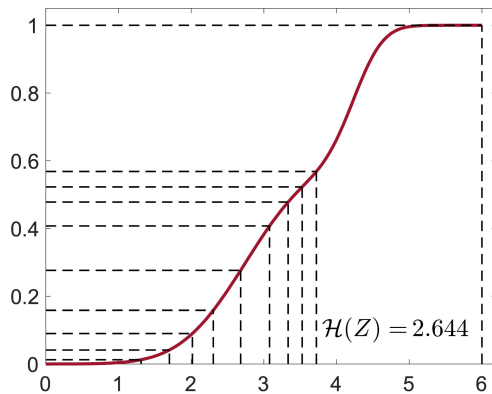
PDF of source and target domains

The target domain is shifted



CDF of source domain

Even division between classes (maximum MI)



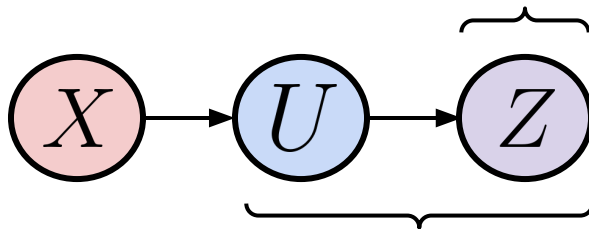
CDF of target domain

MI is no longer maximized

ClusT3: connection to Informantion Theory

Assuming encoded features $U = f(X)$, we find a compressed encoding Z that better preserves information. To do so, we satisfy two conditions:

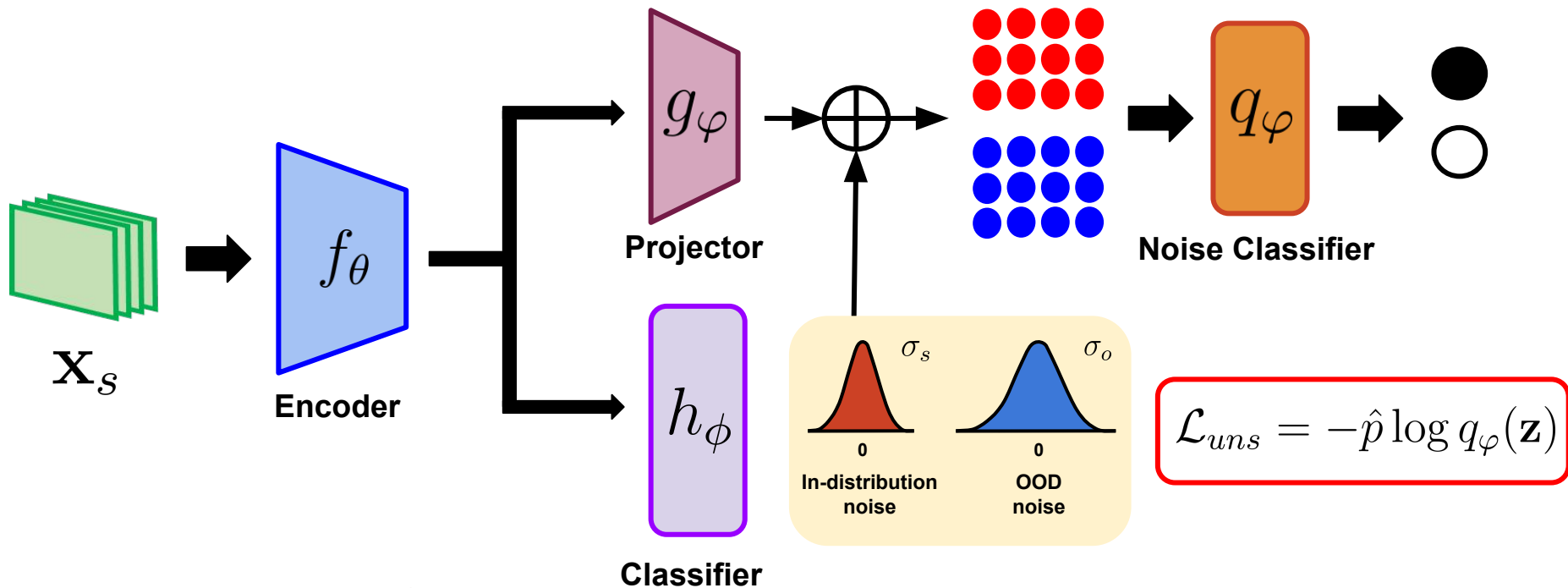
High $\mathcal{H}(Z)$: features space should be split in K regions $\mathcal{R}_k$ with equal probability mass



Low $\mathcal{H}(Z|U)$: Features falling into region $\mathcal{R}_k$ should be similar

NC-TTT: A noise contrastive approach for Test Time Training. CVPR 2024

Learning source distribution by contrasting noisy features:



Without accessing the source domain, we can only learn its distribution by contrasting with something else:

Traditional likelihood estimation

A new sample $\mathbf{z}$ is used to estimate the likelihood from the source distribution.

$$\mathcal{L}_{aux} = -\frac{1}{N_t} \sum_{j=1}^{N_t} \log \hat{p}_s(\mathbf{z}_j)$$

Source distribution (unknown and inaccessible)



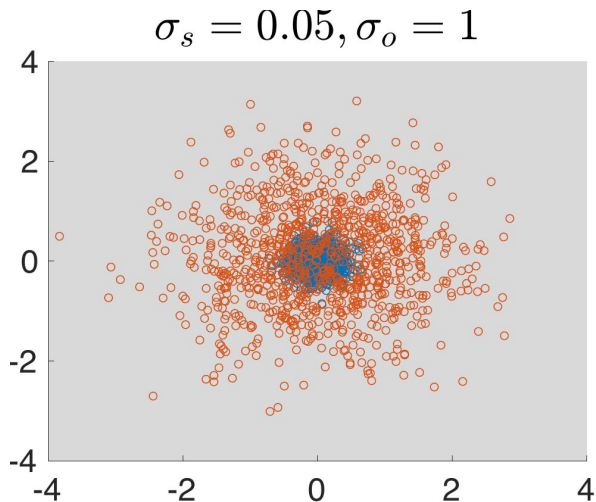
Noise contrastive estimation

In-domain distribution* and Out-of-domain distribution* are used to estimate the likelihood through contrastive learning.

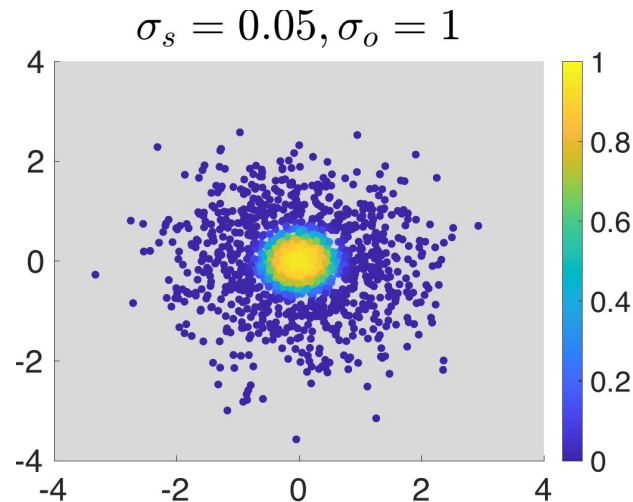
$$p(y_s = 1 | \mathbf{z}) = \frac{\hat{p}_s(\mathbf{z})}{\hat{p}_s(\mathbf{z}) + \hat{p}_o(\mathbf{z})}$$

*Distributions are modelled through kernel density estimation.

We assign a probability to each noise sample:

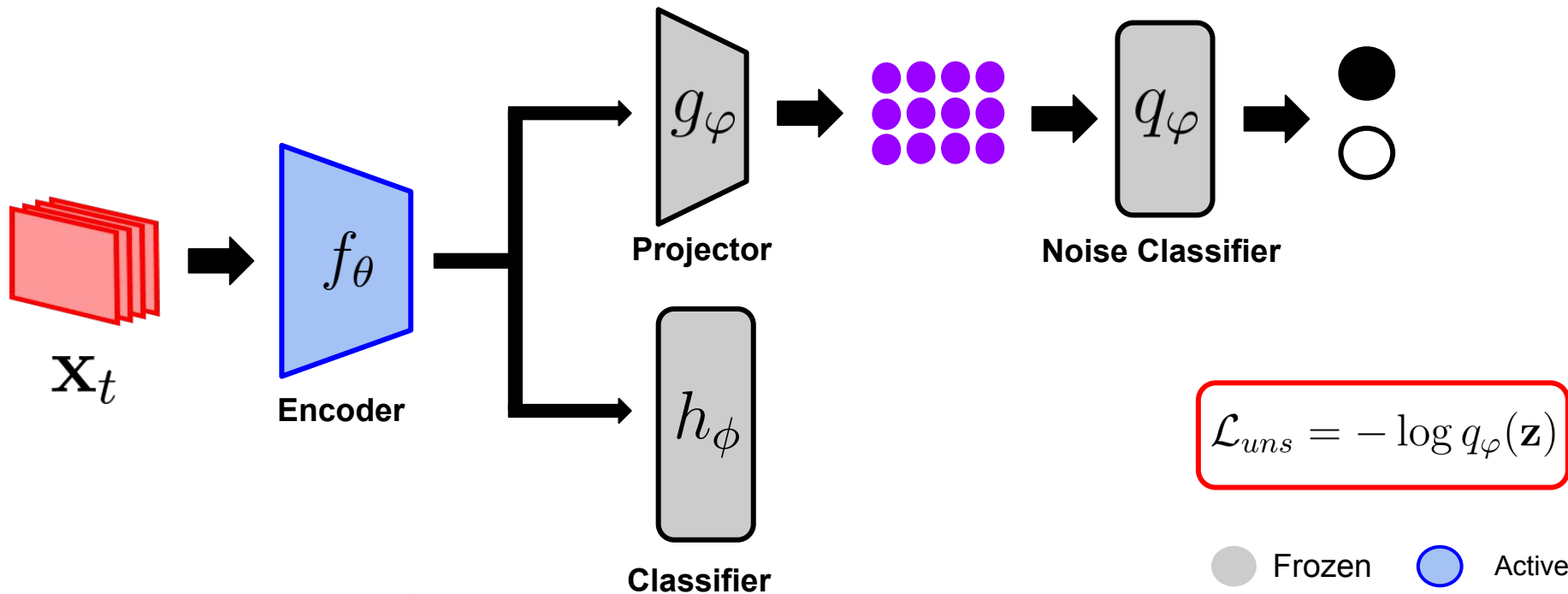


Points from different
distributions overlap as
 $\mu_s = \mu_o = 0$



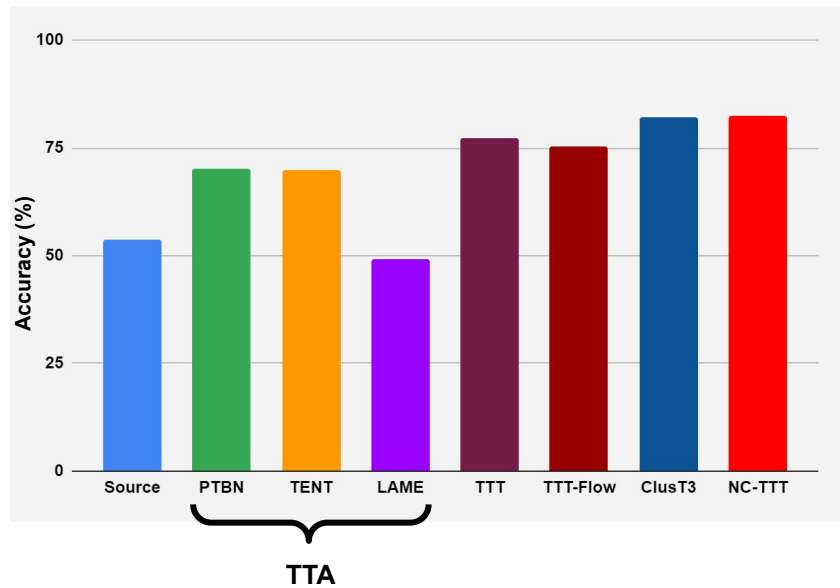
Overlapping overcome
through thresholding

Learning source distribution by contrasting noisy features:

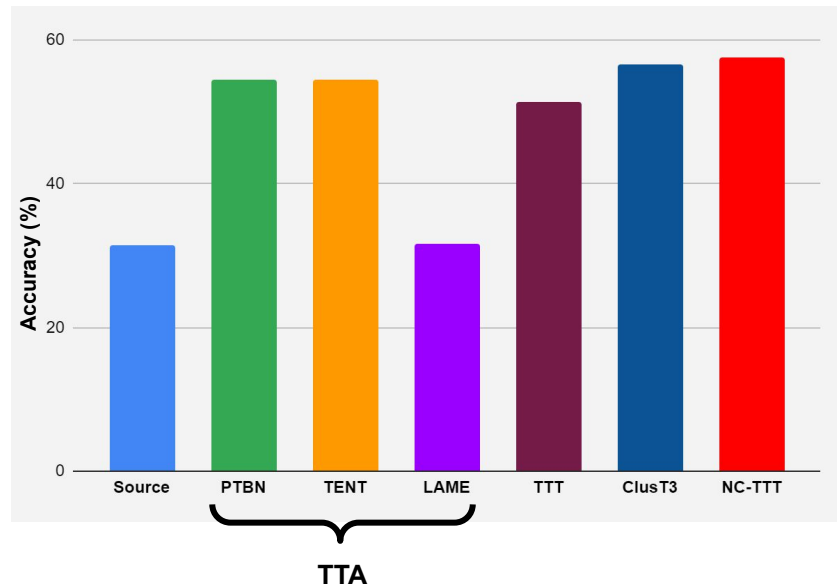


Overall results

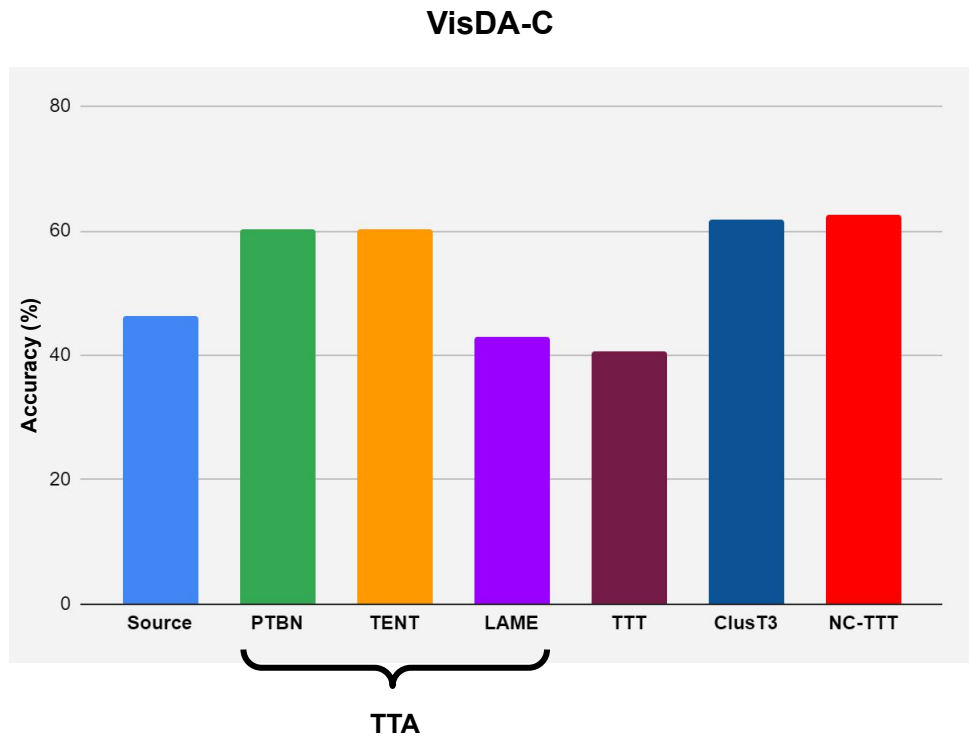
CIFAR-10-C



CIFAR-100-C



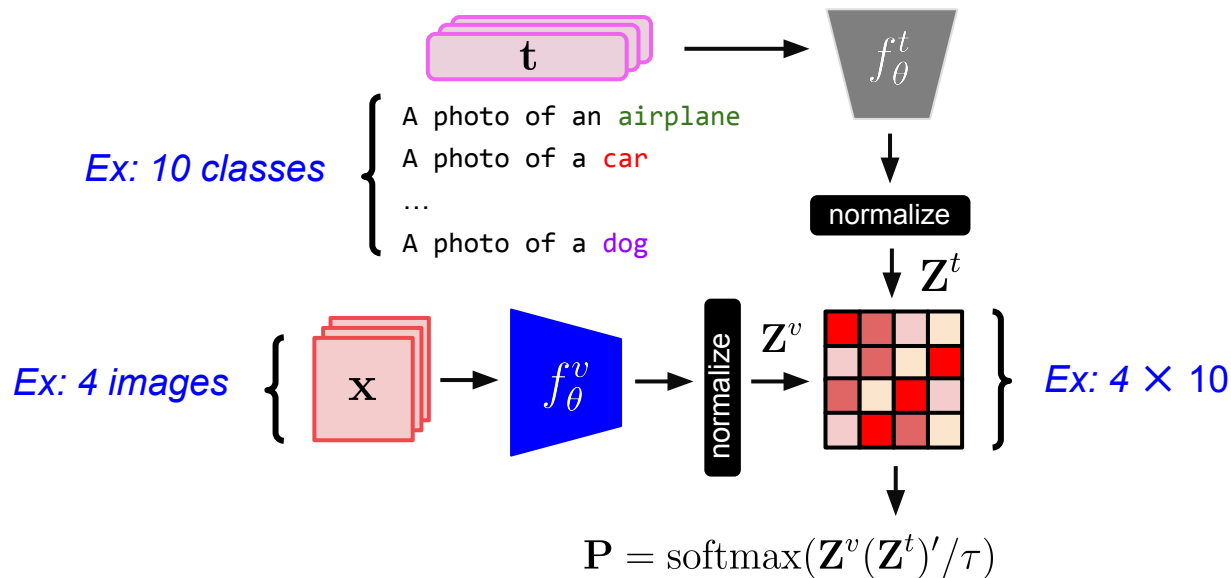
Overall results



CLIPArTT: Light-weight adaptation of CLIP on new domains at test time. ECCV 2024

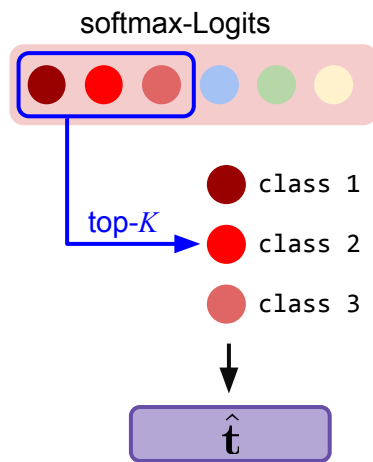
CLIPArTT

Adapting CLIP to new domains. First, obtain predictions:



CLIPArTT

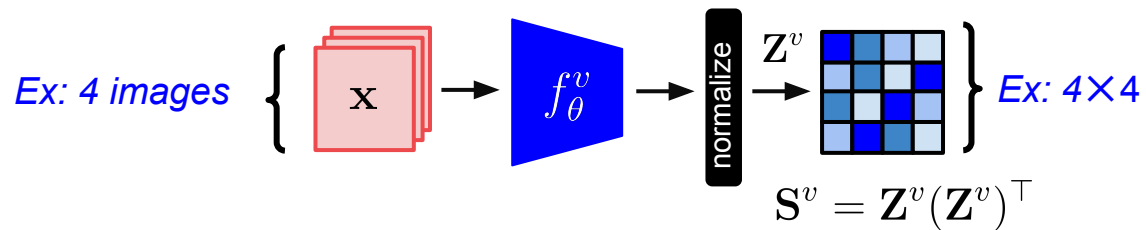
Second, build a new prompt with the top- K classes:



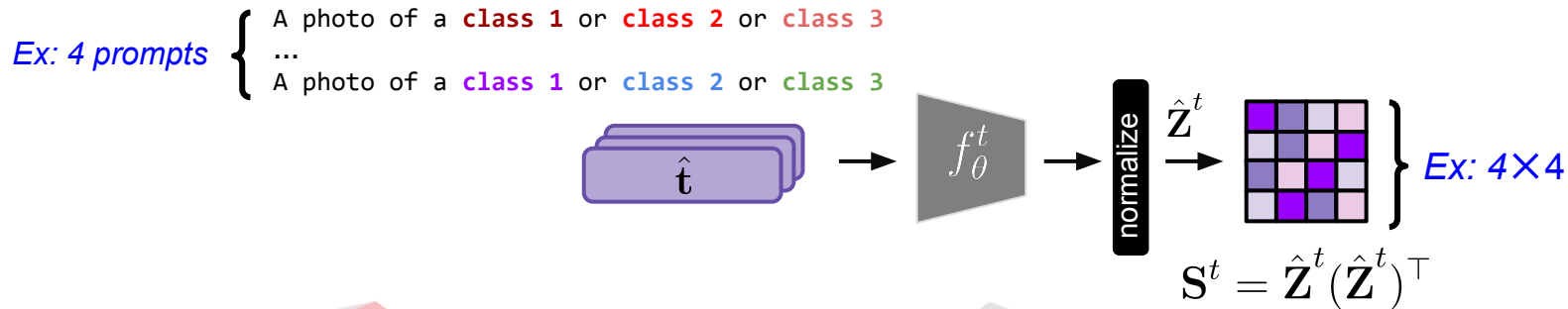
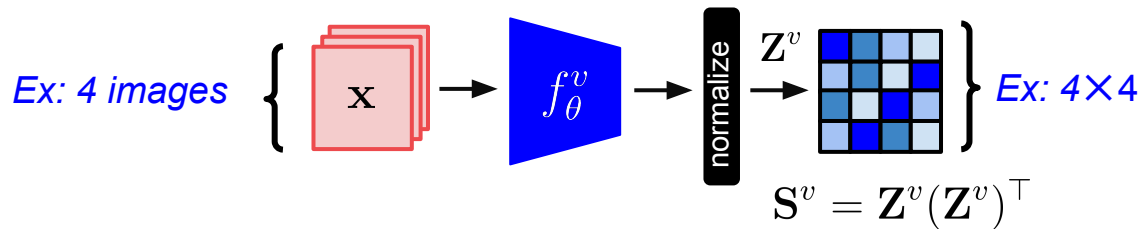
We found that in the majority of cases, the correct class is among the very first softmax logits. We exploit this information.

A photo of a **class 1** or
class 2 or **class 3**

Third, compute **image-to-image**:

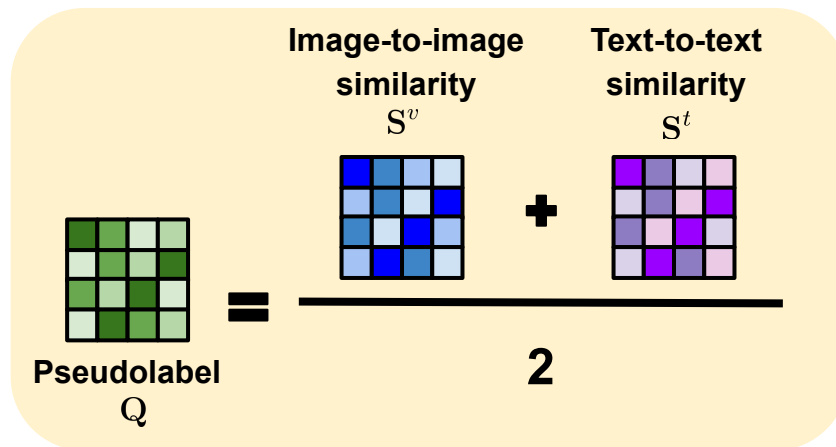


Third, compute **image-to-image** and **text-to-text** similarities:



CLIPArTT

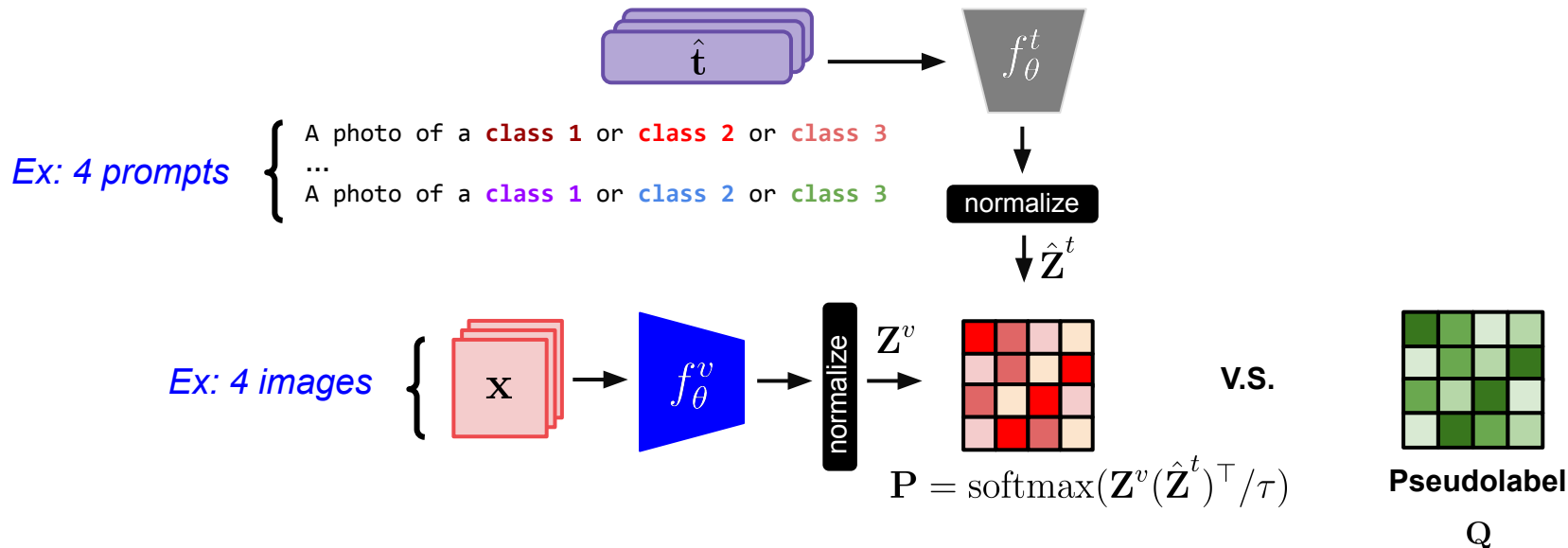
Fourth, build a pseudo-label:



Q contains information that relates every image in the batch with each other, as well as their most probable class predictions with each other.

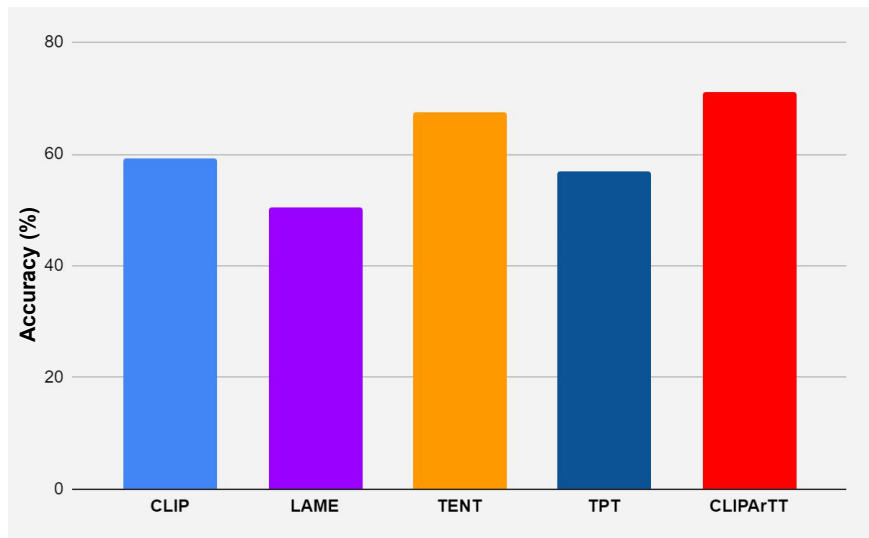
CLIPArTT

Lastly, we use crossentropy to align the image-to-text similarities with the pseudolabels:

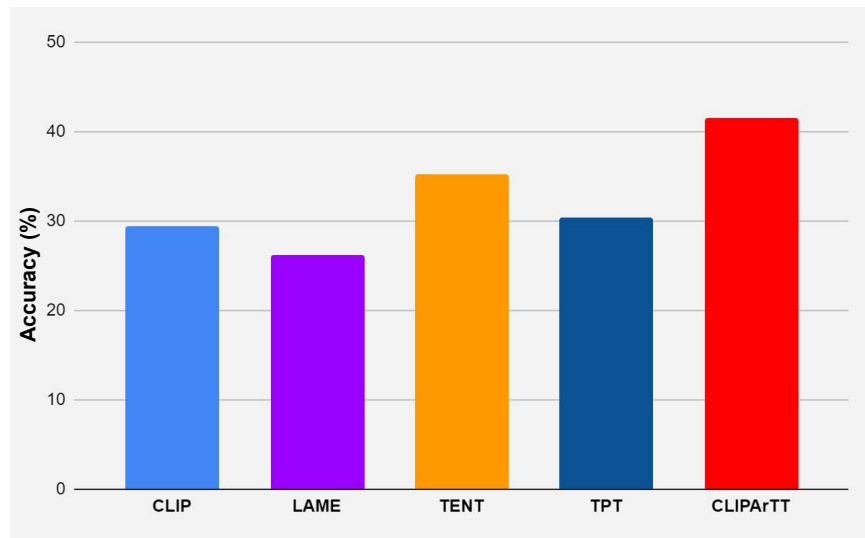


Overall results

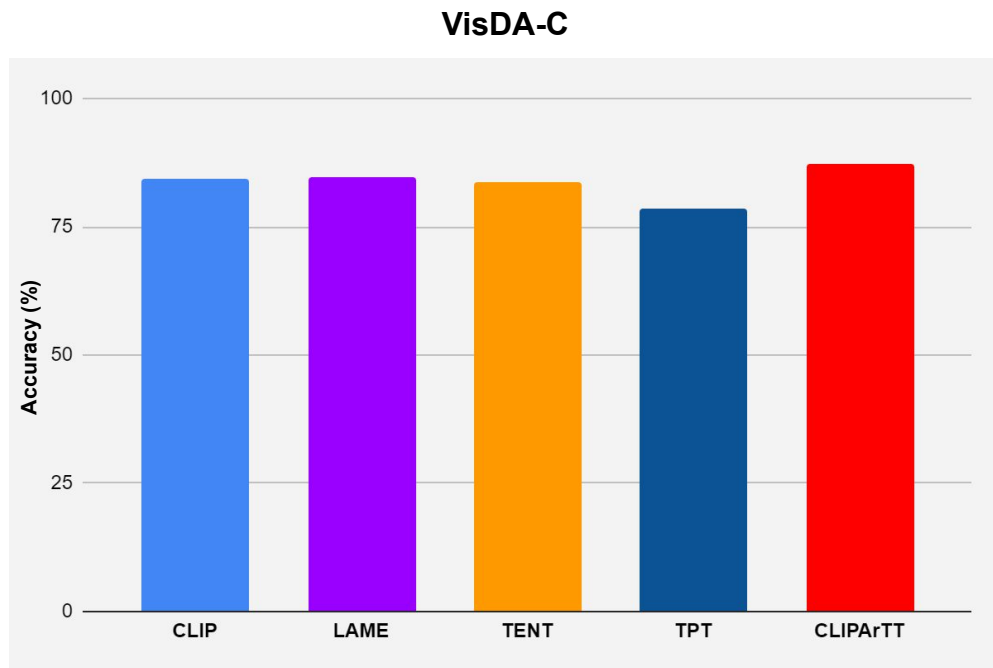
CIFAR-10-C



CIFAR-100-C



Overall results



Future work

- **Holly grail of TTA:** adapting to a single image.
- **TTT without an auxiliary module:** is there a hidden, useful property we can exploit from a model at test-time?
- Variants of the problem: continual, open-set, etc.
- Other challenging tasks: segmentation, object detection, etc.